EXPERIMENT-11:

Write a program for the task of Credit Score Classification

Code:

```
# ============================================================
# CREDIT SCORE CLASSIFICATION
# Random Forest Classifier
# KAGGLE NOTEBOOK
# ============================================================
# 1. Import Libraries
import os
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
print("Searching for CSV files...")
csv_files = []
for root, dirs, files in os.walk("/kaggle/input"):
    for file in files:
        if file.lower().endswith(".csv"):
            csv_files.append(os.path.join(root, file))

print("\nCSV Files Found:")
for file in csv_files:
    print(file)
train_files = [
    file for file in csv_files
    if "train" in os.path.basename(file).lower()
```

```
]
if len(train_files) == 0:
    raise FileNotFoundError(
        "train.csv was not found. Please add your dataset using Add Input."
    )
filename = train_files[0]

print("\nUsing Dataset:")
print(filename)
df = pd.read_csv(filename)
print("\nFirst 5 Rows:")
print(df.head())
print("\nDataset Shape:")
print(df.shape)
print("\nColumn Names:")
print(df.columns.tolist())
remove_columns = [
    "ID",
    "Customer_ID",
    "Name",
    "SSN",
    "Month"
]

for col in remove_columns:
    if col in df.columns:
        df.drop(col, axis=1, inplace=True)
if "Credit_Score" not in df.columns:
    raise ValueError(
        "Credit_Score column was not found in the dataset."
```

```
    )
for col in df.columns:

    if df[col].dtype == "object":

        if df[col].dropna().empty:
            df[col] = df[col].fillna("Unknown")
        else:
            df[col] = df[col].fillna(
                df[col].mode()[0]
            )

    else:
        df[col] = df[col].fillna(
            df[col].median()
        )
encoder = LabelEncoder()
for col in df.columns:
    if df[col].dtype == "object":

        df[col] = encoder.fit_transform(
            df[col].astype(str)
        )
X = df.drop("Credit_Score", axis=1)

y = df["Credit_Score"]
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
```

```
    test_size=0.20,
    random_state=42,
    stratify=y
)
print("\nTraining Data:", X_train.shape)
print("Testing Data :", X_test.shape)
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,

    n_jobs=-1
)
print("\nTraining model...")
model.fit(X_train, y_train)
print("Training completed.")
y_pred = model.predict(X_test)
accuracy = accuracy_score(
    y_test,
    y_pred
)
print("\n========================================")
print("CREDIT SCORE CLASSIFICATION")
print("========================================")
print(
    "Accuracy:",
    round(accuracy * 100, 2),
    "%"
)
```

```
print("\n========================================")
print("CLASSIFICATION REPORT")
print("========================================")

print(
    classification_report(
        y_test,
        y_pred
    )
)
print("\n========================================")
print("CONFUSION MATRIX")
print("========================================")
print(
    confusion_matrix(
        y_test,
        y_pred
    )
)
sample = X.iloc[[0]]
prediction = model.predict(sample)[0]
print("\n========================================")
print("SAMPLE PREDICTION")
print("========================================")
print(
    "Predicted Credit Score:",
    prediction
)
```

Output:

```
Upload the Kaggle train.csv file
Choose Files train[1].csv
train[1].csv(text/csv) - 31136044 bytes, last modified: 8/10/2026 - 100% done
Saving train[1].csv to train[1].csv

Uploaded File: train[1].csv
/tmp/ipykernel_5562/3659475921.py:32: DtypeWarning: Columns (26) have mixed types. Specify dtype option on import or set low_memory=False.
  df = pd.read_csv(filename)

First 5 Rows:
      ID Customer_ID     Month           Name   Age          SSN Occupation  \
0  0x1602   CUS_0xd40   January  Aaron Maashoh    23  821-00-0265  Scientist
1  0x1603   CUS_0xd40  February  Aaron Maashoh    23  821-00-0265  Scientist
2  0x1604   CUS_0xd40     March  Aaron Maashoh  -500  821-00-0265  Scientist
3  0x1605   CUS_0xd40     April  Aaron Maashoh    23  821-00-0265  Scientist
4  0x1606   CUS_0xd40       May  Aaron Maashoh    23  821-00-0265  Scientist

  Annual_Income  Monthly_Inhand_Salary  Num_Bank_Accounts  ...  Credit_Mix  \
0      19114.12            1824.843333                  3  ...           _
1      19114.12                    NaN                  3  ...        Good
2      19114.12                    NaN                  3  ...        Good
3      19114.12                    NaN                  3  ...        Good
4      19114.12            1824.843333                  3  ...        Good

   Outstanding_Debt Credit_Utilization_Ratio     Credit_History_Age  \
0            809.98                26.822620  22 Years and 1 Months
1            809.98                31.944960                    NaN
2            809.98                28.609352  22 Years and 3 Months
3            809.98                31.377862  22 Years and 4 Months
4            809.98                24.797347  22 Years and 5 Months
```

```
   Outstanding_Debt Credit_Utilization_Ratio     Credit_History_Age  \
0            809.98                26.822620  22 Years and 1 Months
1            809.98                31.944960                    NaN
2            809.98                28.609352  22 Years and 3 Months
3            809.98                31.377862  22 Years and 4 Months
4            809.98                24.797347  22 Years and 5 Months

   Payment_of_Min_Amount Total_EMI_per_month Amount_invested_monthly  \
0                     No           49.574949       80.41529543900253
1                     No           49.574949      118.28022162236736
2                     No           49.574949         81.699521264648
3                     No           49.574949       199.4580743910713
4                     No           49.574949      41.420153086217326

                  Payment_Behaviour     Monthly_Balance Credit_Score
0   High_spent_Small_value_payments  312.49408867943663         Good
1    Low_spent_Large_value_payments  284.62916249607184         Good
2   Low_spent_Medium_value_payments   331.2098628537912         Good
3    Low_spent_Small_value_payments  223.45130972736786         Good
4  High_spent_Medium_value_payments  341.48923103222177         Good

[5 rows x 28 columns]

Dataset Shape:
(100000, 28)

Column Names:
['ID', 'Customer_ID', 'Month', 'Name', 'Age', 'SSN', 'Occupation', 'Annual_Income', 'Monthly_Inhand_Salary', 'Num_Bank_Accounts',

Training Data: (80000, 22)
Testing Data : (20000, 22)
```

```
Training model...
Training completed.

==================================
CREDIT SCORE CLASSIFICATION
==================================
Accuracy: 78.15 %

==================================
CLASSIFICATION REPORT
==================================
              precision    recall  f1-score   support

           0       0.74      0.71      0.72      3566
           1       0.77      0.78      0.78      5799
           2       0.80      0.80      0.80     10635

    accuracy                           0.78     20000
   macro avg       0.77      0.77      0.77     20000
weighted avg       0.78      0.78      0.78     20000


==================================
CONFUSION MATRIX
==================================
[[2528   26 1012]
 [ 108 4545 1146]
 [ 781 1297 8557]]
```

```
==================================
SAMPLE PREDICTION
==================================
Predicted Credit Score: 0
```